

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMATICA



INFORME TECNICO

MATERIA: PROGRAMACION MULTIMEDIAL

DOCENTE: PH.D. MOISES MARTIN SILVA CHOQUE

INTEGRANTES:

- CHINO NINA OSCAR
- MAMANI BAUTISTA WILSON
- MACHACA CALLISAYA JOSELIN GLADYS
- ZAPATA GUTIÉRREZ JHORDAN FREDDY

ACTIVIDADES GRUPALES

A. Digitalización de Trámites Universitarios (UMSA)

METODOLOGÍA UTILIZADA

Para la construcción de este módulo se aplicó el enfoque de **Arquitectura Guiada por Eventos** en el desarrollo Frontend, asegurando que cada interacción del usuario se valide bajo estrictas reglas de negocio antes de tocar el servidor:

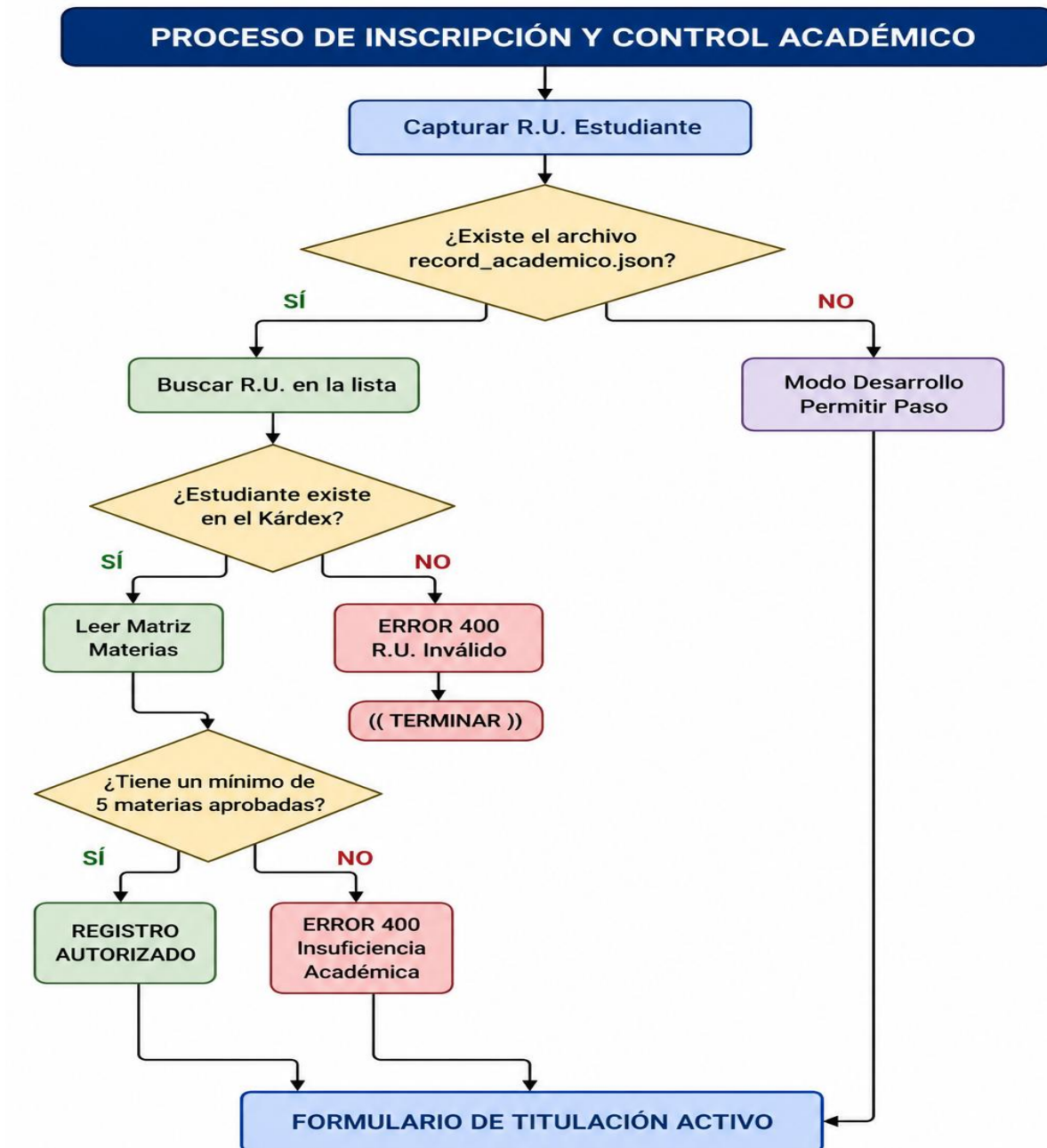
1. **Carga Simultánea y Cruce de Datos:** Al iniciar la página, se ejecuta una petición paralela (Promise.all) a los archivos del servidor para cruzar los prerequisites de la malla curricular con el historial de aprobadas del alumno.
2. **Evaluación de Estados en Tiempo Real:** Se implementó una máquina de estados para la renderización de la tabla. Cada materia pasa por una evaluación de tres fases:
 - Fase disponible
 - Fase seleccionada
 - Fase inscrita/ bloqueada
3. **Persistencia e Inmutabilidad Local:** Una vez confirmado el trámite, los datos se escriben en el cliente de manera persistente (localStorage) para congelar la interfaz, mitigando el riesgo de que el usuario envíe solicitudes duplicadas a la API si recarga la página.

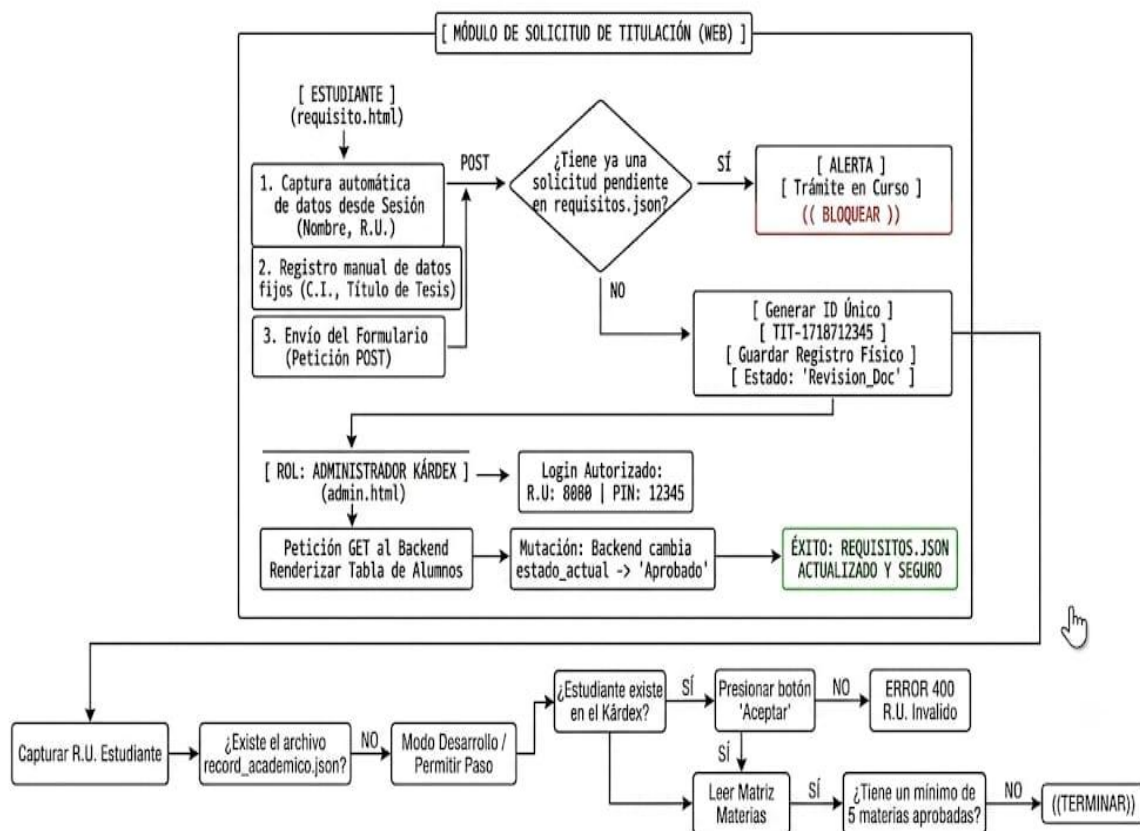
2. HERRAMIENTAS Y TECNOLOGÍAS EMPLEADAS

- **Estructura y Estilos (HTML5 / CSS3):** Contenedores dinámicos, tablas de datos académicas y capas superiores modales (display: flex) para ventanas emergentes de confirmación.
- **Lógica del Cliente (JavaScript ES6):** Manejo asíncrono (fetch, async/await), manipulación del DOM, iteradores de arreglos (map, filter, forEach) y gestión de memoria del navegador (localStorage).

- **Intercambio de Datos (JSON):** Estructuras limpias para la transferencia del payload (envío del ru y el arreglo de materias) hacia el backend.

3.FLUJOS DE PROCESOS

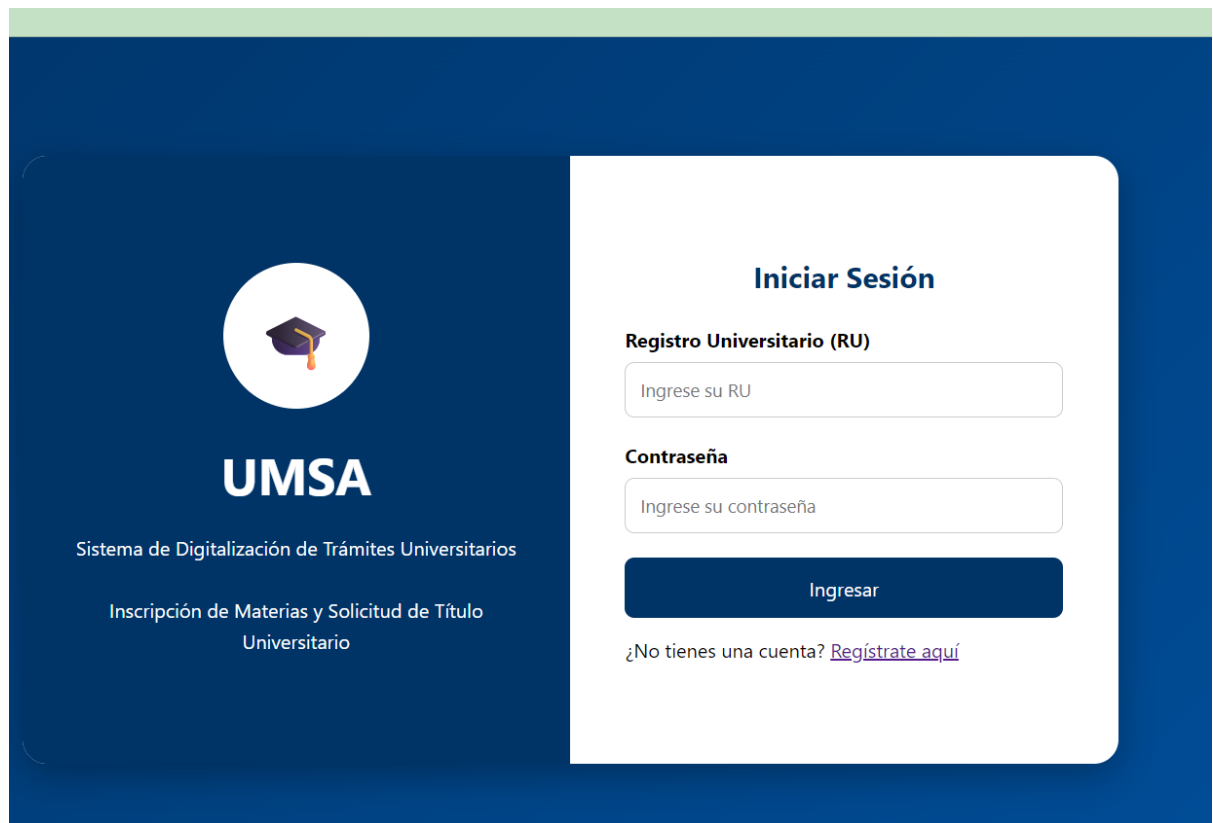




4.CAPTURAS DE PANTALLA O RESULTADOS OBTENIDOS

A. Flujo del Estudiante (Frontend -> Backend)

1. **Carga Dinámica:** El archivo `requisito.html` extrae los datos de sesión activa del `localStorage` bloqueando los campos institucionales (`Nombre`, `R.U.`) para evitar su alteración.
2. **Input de Datos Críticos:** El alumno registra únicamente campos variables en texto plano: Cédula de Identidad (`ci`) y el Título oficial de su Tesis o Proyecto de Grado (`proyecto`).
3. **Control de Concurrencia Duplicada:** El backend verifica dentro de `requisitos.json` que el R.U. no posea un trámite previo en estado activo. Si pasa el filtro, se genera un identificador único globalizado bajo el estándar Unix Timestamp (`TIT-171871XXXXXX`).
4. **Persistencia:** La solicitud se guarda con el estado inicial `'Revision_Documentos'`.



```
EXPLORER    ... dashboard.html    < inscripcion.html    < titulo.html    < estado.html    < index.html .\ A

▼ BMP
  > BPM_Tramites
  ▼ data
    {} estudiantes.json
    {} inscripciones.json
    {} materias.json
    {} record_academico.json
    {} titulos.json
  > Fotogrametria
  > node_modules
  ▼ public
    < dashboard.html
    < estado.html
    < index.html
    < inscripcion.html
    < registro.html
    < titulo.html
  > Unity_Animacion
  < index.html
  {} package-lock.json
  {} package.json

data > {} estudiantes.json > {} 2
1  [
2  {
3    "ru": "2021",
4    "nombre": "Juan Pérez",
5    "carrera": "Informática",
6    "correo": "[juan.perez@umsa.bo](mailto:juan.perez@umsa.bo)",
7    "password": "12345"
8  },
9  {
10   "ru": "2022",
11   "nombre": "María López",
12   "carrera": "Ingeniería de Sistemas",
13   "correo": "[maria.lopez@umsa.bo](mailto:maria.lopez@umsa.bo)",
14   "password": "12345"
15 },
16 ],
17 {
18   "ru": "2023",
19   "nombre": "Carlos Quispe",
20   "carrera": "Estadística",
21   "correo": "[carlos.quispe@umsa.bo](mailto:carlos.quispe@umsa.bo)",
22   "password": "12345"
23 },
24 ]
```

B. Persistencia Posterior a la Confirmación

Tras recibir un respuesta.ok del backend, las materias seleccionadas se integran al historial de la sesión del usuario. El botón se bloquea y se transforma automáticamente en un indicador pasivo de que el proceso ha sido despachado.

Datos del Estudiante

Nombre: Juan Pérez

RU: 2021

Carrera: Informática

Récord Académico

Código	Materia	Nota	Estado
INF119	Programación I	85	 Aprobada
INF120	Programación II	78	 Aprobada
INF210	Estructuras de Datos	90	 Aprobada
INF220	Arquitectura de Computadoras	82	 Aprobada
INF223	Matemática Discreta	75	 Aprobada
INF318	Sistemas Operativos	88	 Aprobada
INF310	Análisis Numérico	79	 Aprobada

BOLETA OFICIAL DE INSCRIPCIÓN

Estudiante: María López

R.U.: 2022

Carrera: Ingeniería de Sistemas

Verificación: Sistema de Kardex Digital UMSA

Asignaturas Consolidadas:

-  **INF220** - Arquitectura de Computadoras (**Registrado**)
-  **INF223** - Matemática Discreta (**Registrado**)

Esta boleta es un reflejo de los datos validados y persistidos en el sistema para el periodo actual.

 Imprimir / Guardar PDF

Anterior

C. Flujo del Administrador (Kárdex Académico)

1. **Autenticación Dedicada:** El formulario de ingreso evalúa credenciales estricta y directamente en los controladores del sistema:
 - **R.U. Autorizado:** 8080
 - **Contraseña Encriptada/Validada:** 12345
2. **Redirección Estructural:** Al validar el rol de auditor, la interfaz redirige forzosamente hacia `admin.html`.
3. **Consumo de API Rest:** El panel ejecuta una petición asíncrona `GET /admin/solicitudes` mapeando dinámicamente el archivo `requisitos.json` en una tabla de control visual.
4. **Mutación de Estado:** Al accionar el botón "Aceptar", se efectúa una petición `POST /admin/cambiar-estado`. El backend busca el `id_tramite` específico y reescribe el campo fundamental a 'Aprobado', completando el ciclo de vida del trámite.

 **Verificación Académica**

✓ ¡Expediente Válido! Has aprobado las 17 materias del plan curricular.

Postulante: María López

Carrera: Ingeniería de Sistemas

R.U.: 2022

Anterior

Siguiente



Formulario de Requisitos de Titulación

Por favor, confirme sus datos institucionales y registre el título oficial de su modalidad de graduación.

Nombre Completo:

Maria López

Registro Universitario (R.U.):

2022

Cédula de Identidad (C.I.):

Ej. 8432114 LP

Título de la Tesis / Proyecto:

Ej. Criptografía caótica aplicada a imágenes digitales

Enviar Solicitud

Anterior



Formulario de Requisitos



Estado: Solicitud Pendiente de Revisión

Tu formulario de registro de modalidad ya fue enviado al Kárdex Académico.
Por favor, espera a que un administrador evalúe los datos y otorgue el visto bueno.

Volver al Menú

Panel de Control de Titulación - Kárdex

Listado de solicitudes de modalidad registradas en el sistema:

R.U.	Estudiante	C.I.	Título del Proyecto / Tesis	Estado Actual	Acción
2021	Juan Pérez	543534	"Programacion cuantica"	Pendiente	<button>Aceptar</button>
2022	María López	42343	"Programacion cuantica"	Pendiente	<button>Aceptar</button>

FLUJO

Division de Titulos y Diplomas UMSA

 Verificación Académica

✖ Trámite Bloqueado: No cumples con los requisitos de egreso.

Para iniciar la modalidad de titulación debes concluir el plan de estudios.

Materias Aprobadas: 7 de 17
Materias Faltantes: 10 asignaturas.

Volver al Menú

B. Desarrollo 3D y Realidad Virtual

a) Animación Sincronizada en Unity

1. Introducción y Objetivos

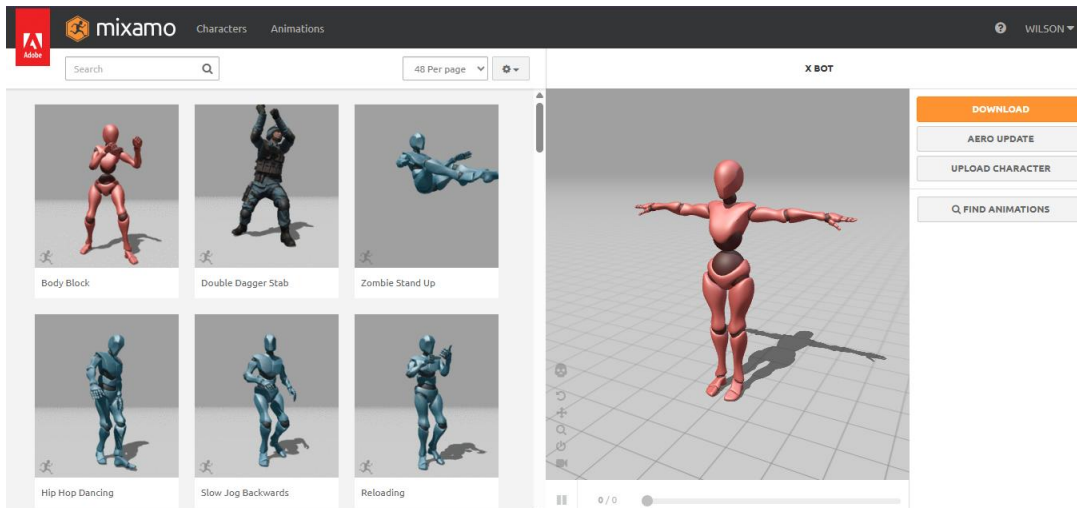
Introducción: Desarrollo de una escena 3d interactiva en la que múltiples personajes virtuales ejecutan una rutina de coreografía sincronizada con una pista de audio, diseñada para su ejecución en navegadores web.

2. Herramientas y Recursos

- **Motor Gráfico:** Unity 6 (versión 6000.3.17f1 LTS) con el módulo *Web Build Support* instalado.
- **Recursos 3D (Assets):** Plataforma Mixamo para la obtención de los modelos de personajes (.fbx) y sus respectivas animaciones de baile.
- **Lenguaje de Programación:** C# utilizado para la elaboración de scripts donde maneja el control de cámara y algunas luces.
- **Audio:** Pista en formato .mp3 gestionada mediante el componente AudioSource.

3. Procesamiento en Unity

Paso 1: Configuración de Modelos Se importaron los archivos. fbx y se modificó su tipo de animación (Animation Type) a **Humanoid** para estandarizar el esqueleto.



Posteriormente, se creó un Animator Controller ("ControladorBaile") asignando la animación principal. *Nota técnica:* Se desactivó la propiedad "Has Exit Time" en las transiciones del Animator para que el cambio del estado de reposo al estado de baile fuera completamente instantáneo y no generara latencia.

Paso 2: desarrollo del scripts Para evitar el desajuste entre los tiempos de carga de los diferentes modelos, se prescindió del inicio automático y se desarrolló el script CoreografíaController.cs. Este código rastrea dinámicamente todos los componentes Animator en la escena mediante la función FindObjectsOfType. Al iniciar, el script congela el tiempo de los personajes (`anim.speed = 0f`). Al recibir la orden de inicio, el código libera la velocidad y fuerza el estado de animación (`mixamo_com`) simultáneamente a la reproducción del AudioSource.

Paso 3: Interfaz de Usuario y exportación de WebGL Los navegadores web modernos bloquean la reproducción automática de audio, se implementó un Canvas con un componente Button (TextMeshPro). Este botón funciona como el disparador del script central, haciendo la ejecución del código.

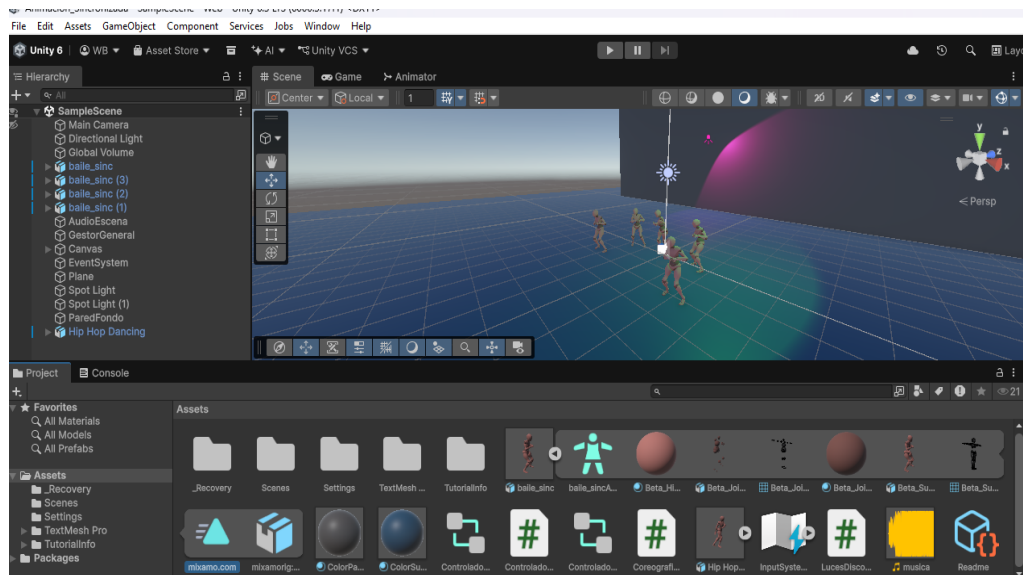
4. Post-procesamiento y Diseño

Una vez lograda la sincronización, se procedió a refinar la calidad visual y la interactividad de la escena:

- **Escenario:** Se insertó un plano (Plane) como suelo y un cubo escalado como pared de fondo, a los cuales se les aplicó un material oscuro (casi negro) para mejorar el contraste. Se desactivó el Skybox por defecto cambiándolo a un fondo de color sólido.
- **Iluminación Dinámica:** Se añadieron dos luces direccionales tipo Spotlight (colores fucsia y verde) sobre los personajes. Se desarrolló el script LucesDiscoteca.cs utilizando la función matemática `Mathf.PingPong` para interpolar la intensidad de las luces, generando un efecto de titileo rítmico.
- **Cámara Libre:** Se integró el script ControladorCamara.cs a la Main Camera, permitiendo el mapeo de los ejes de entrada del teclado para navegar por la escena en tiempo real usando las teclas W, A, S, D, Q y E.
- **Exportación WebGL:** En la ventana *Build Profiles*, se desactivó la compresión (*Compression Format: Disabled*) para prevenir errores de desempaquetado en el navegador, generando exitosamente los archivos de datos y el `index.html`.

5. Resultados y Capturas de Pantalla

- (poner captura de la cuadrícula del Animator donde se vea el estado "Quieto" y el estado "mixamo_com" sin la flecha de Has Exit Time)



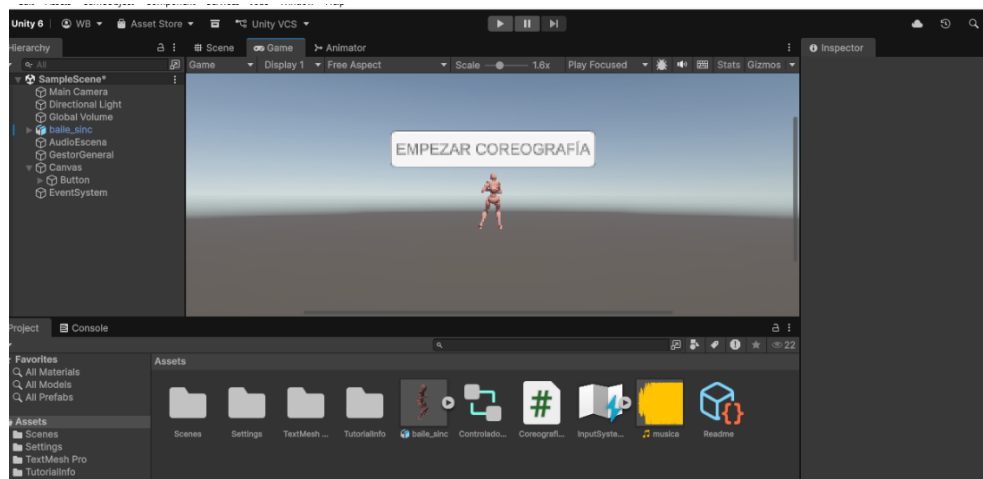
```

C# CoreografiaController.cs
Assets > C# CoreografiaController.cs > CoreografiaController > IniciarCoreografia() : void
1 using UnityEngine;
2 0 references
3 public class CoreografiaController : MonoBehaviour
4 {
5     1 reference
6     public AudioSource musica;
7
8     0 references
9     void Start()
10 {
11     Animator[] todosLosAnimators = FindObjectsOfType<Animator>();
12     foreach (Animator anim in todosLosAnimators)
13     {
14         anim.speed = 0f;
15     }
16
17     0 references
18     public void IniciarCoreografia()
19 {
20     musica.Play();
21     Animator[] todosLosAnimators = FindObjectsOfType<Animator>();
22     foreach (Animator anim in todosLosAnimators)
23     {
24         if (anim != null && anim.runtimeAnimatorController != null)
25         {
26             anim.speed = 1f;
27             anim.Play("mixamo_com", -1, 0f);
28         }
29     }
30 }

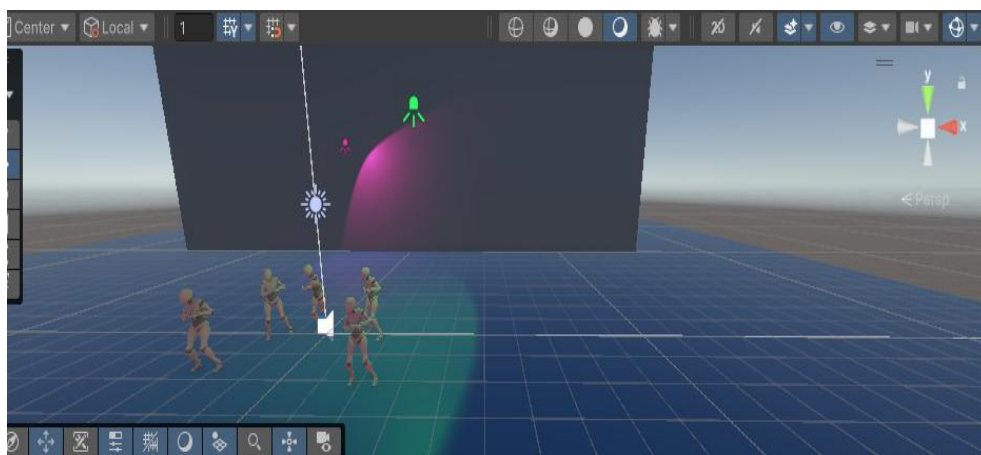
```

Comparación Visual del Entorno ("Antes" y "Después"):

- **Antes del diseño de nivel:** La escena mostraba un único personaje estático, con el tamaño de cámara por defecto, flotando sobre un fondo gris/azul sin texturas y con iluminación plana.



- **Después del diseño de nivel:** El proyecto final presenta un grupo de baile completo en posición de "V", con un entorno cerrado de alto contraste, iluminación de escenario dinámica coloreada y controles de cámara ajustados.



FOTOGRAMETRÍA DIGITAL

1. Introducción y Objetivos

Reconstrucción tridimensional (3D) de una persona mediante el procesamiento de fotogramas de video.

Objetivos:

- Generar una nube de puntos 3D mediante técnicas de fotogrametría digital utilizando el software COLMAP.
- Visualizar, limpiar y optimizar la nube de puntos en formato (.ply) mediante el software MeshLab.

2. Herramientas

- Dispositivo Móvil: **Redmi A2 para la grabación del video fuente.**
- **Google Colab (Python):** Utilizado para el procesamiento del video y la extracción automatizada de fotogramas secuenciales.
- **COLMAP:** Software de fotogrametría encargado de la orientación de cámaras y generación de la nube de puntos.

- **MeshLab:** Herramienta de procesamiento de mallas y nubes de puntos 3D para la edición, limpieza y optimización del diseño final.

Estrategia de Toma

Se realizó una grabación de video continuo rodeando a la persona en un ángulo de **360 grados**. Se mantuvo una velocidad constante y una distancia uniforme para asegurar un **solapamiento (overlap) de aproximadamente el 70%** entre cada fotograma, garantizando así la redundancia visual necesaria para el algoritmo.

- **Cantidad de datos:** Se extrajeron e importaron al proyecto entre **500 y 600 fotogramas** en total, obtenidas con el siguiente código:

```
from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

archivo='/content/drive/MyDrive/data/VideoJ.mp4'

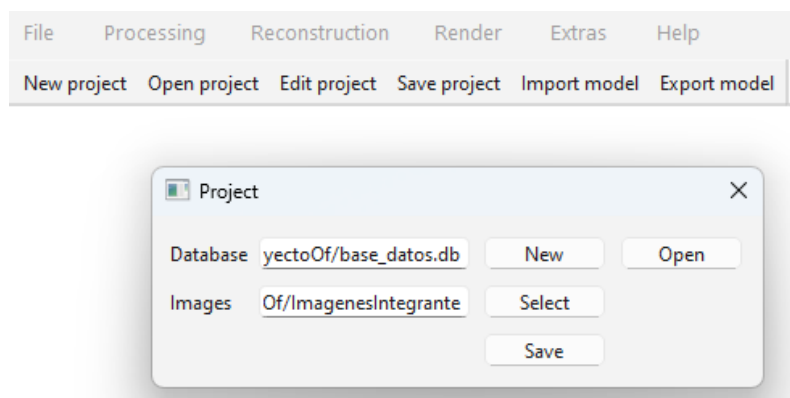
import cv2

video=cv2.VideoCapture(archivo)

contador=1
while True:
    retorno, frame=video.read()
    if not retorno:
        break;
    cv2.imwrite('/content/drive/MyDrive/data/ImagenOf/imagen'+str(contador)+'.jpg', frame)
    contador=contador+1
```

3. Procesamiento en COLMAP (Fotogrametría)

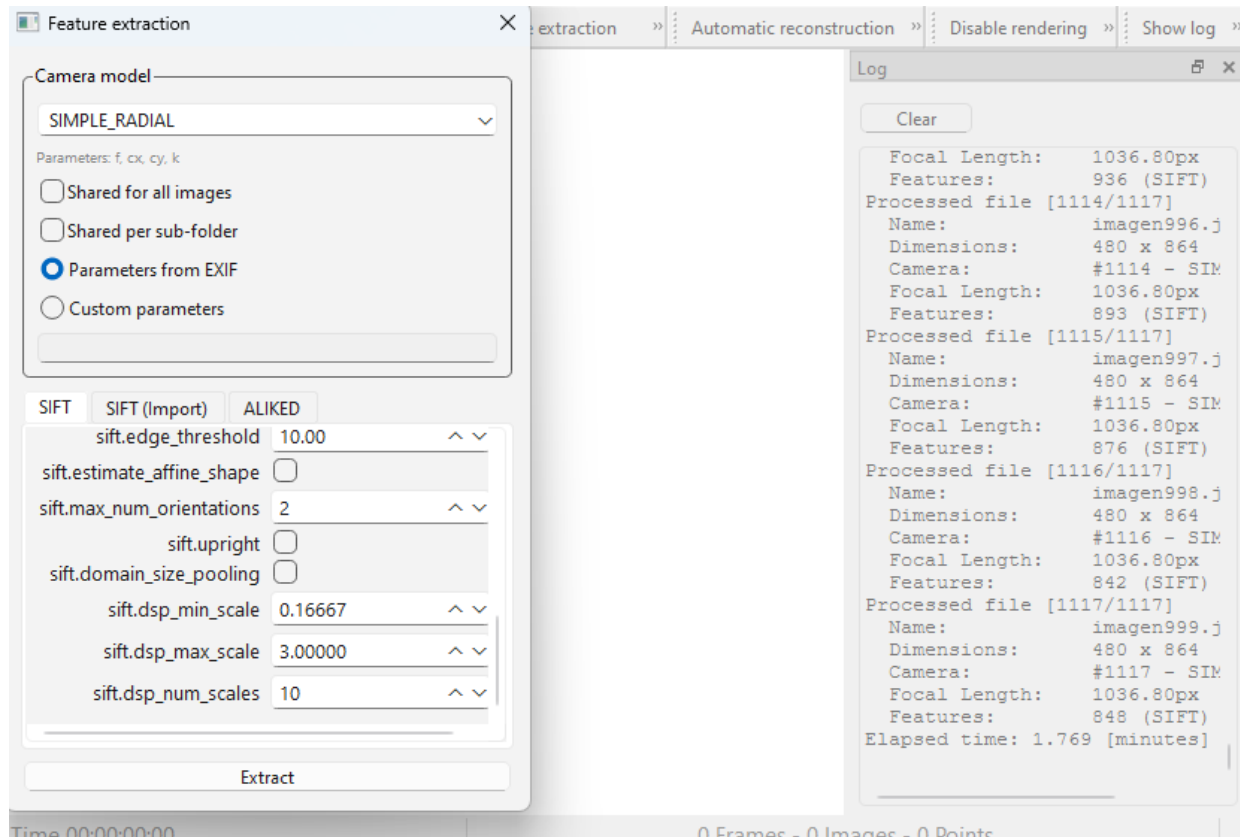
Para iniciar el proyecto en la interfaz gráfica de COLMAP, se ejecutó la secuencia: File → New Project. En la ventana emergente, se configuró la ruta del proyecto creando una nueva base de datos (.db) y se seleccionó la carpeta contenedora de las imágenes obtenidas en Google Colab mediante el botón Select.



Paso 1: Extracción de Características (*Feature Extraction*)

El software analiza cada imagen de forma individual mediante algoritmos de visión artificial para detectar puntos clave o de interés (esquinas, texturas y bordes únicos).

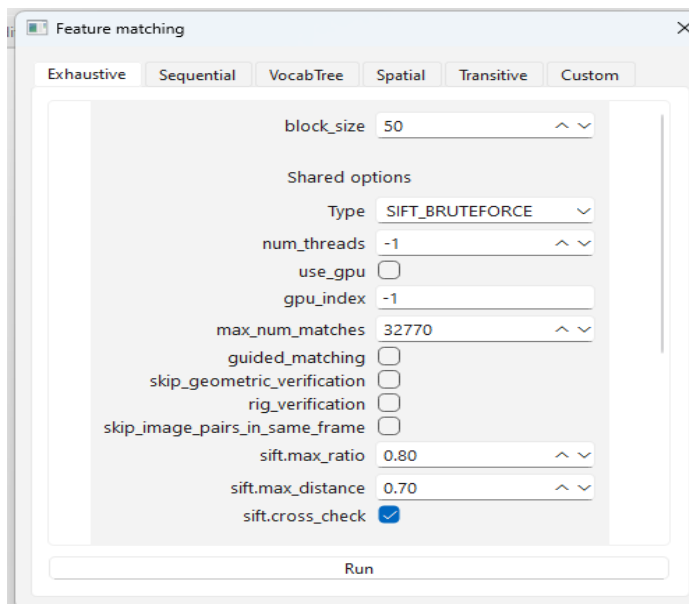
Nota técnica: Se configuró el uso de aceleración por hardware (GPU) activando la casilla **CUDA** (en caso de contar con tarjeta gráfica NVIDIA) para optimizar el tiempo de cómputo antes de presionar Extract.



Paso 2: Emparejamiento de Características (*Feature Matching*)

COLMAP compara los puntos clave detectados entre todas las imágenes del conjunto para encontrar coincidencias y determinar la relación espacial de las fotos.

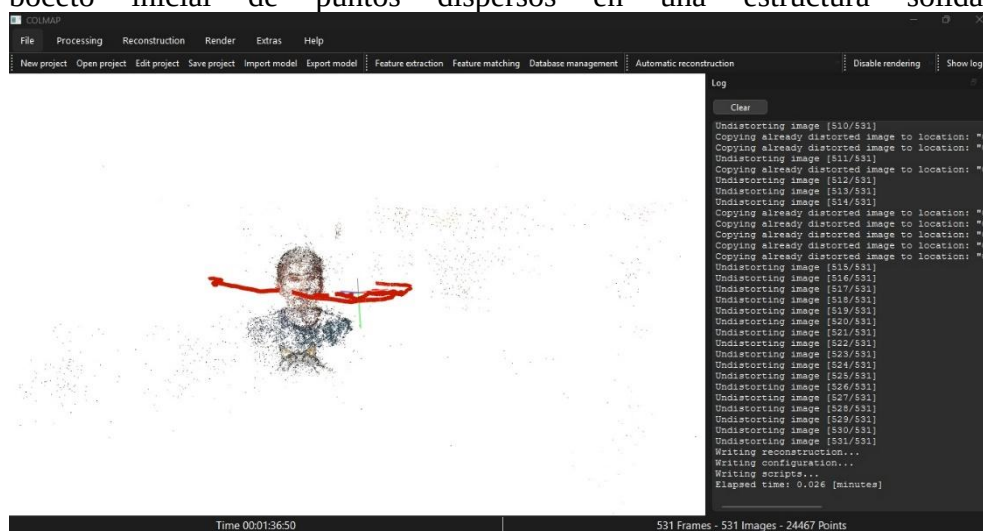
- **Ruta:** Processing → Feature Matching.
- **Criterio:** Al tratarse de fotogramas extraídos de un video continuo y ordenado, se seleccionó el emparejamiento de tipo **Sequential** (Secuencial). *[Si las fotos hubieran sido tomadas al azar, se habría optado por Exhaustive (Exhaustivo)]*. Finalmente, se ejecutó con el botón Run.



Paso 3: Reconstrucción Dispersa (*Sparse Reconstruction*)

El programa ejecuta el algoritmo *Structure from Motion* (SfM) para calcular la posición e inclinación exacta de la cámara al momento de cada toma, generando la primera estructura geométrica flotante.

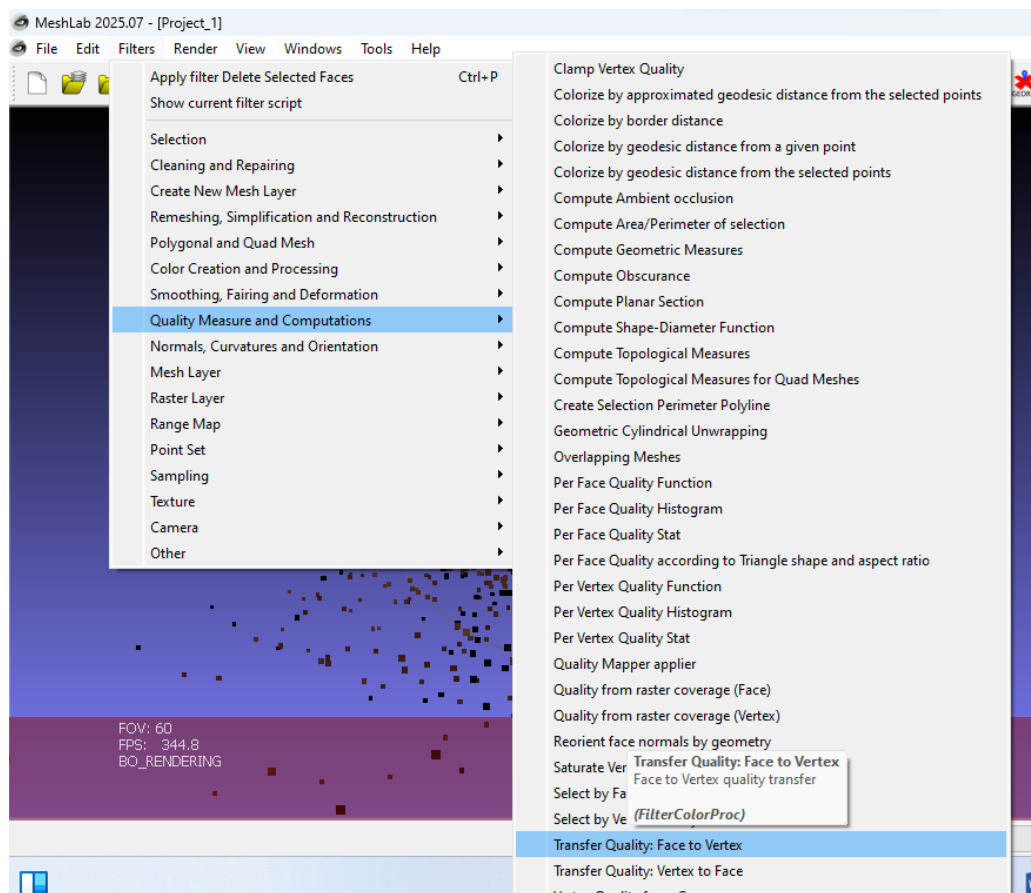
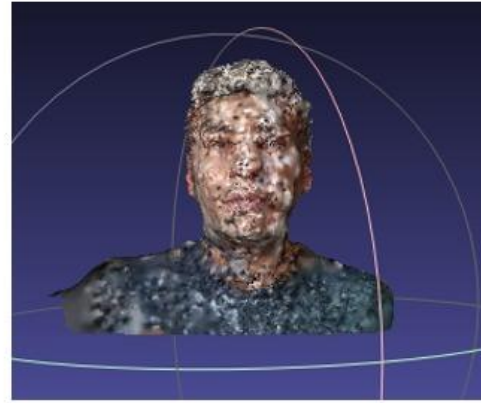
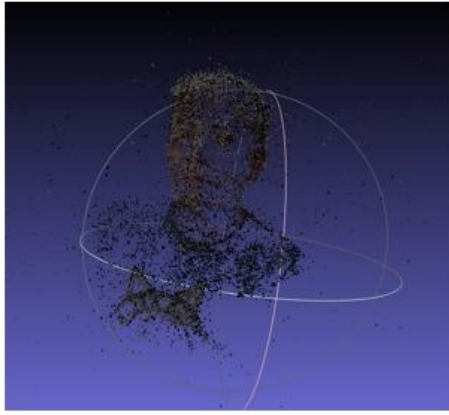
- **Ruta:** En la pestaña Reconstruction, se optó por la reconstrucción manual mediante Start Reconstruction para visualizar en tiempo real el progreso de alineación y la aparición de los puntos en pantalla.
- **Paso Opcional (Reconstrucción Densa):** Si se dispone de soporte CUDA nativo, se procedió a ejecutar el comando Dense Reconstruction para densificar la geometría, transformando el boceto inicial de puntos dispersos en una estructura sólida y detallada.



4. Post-procesamiento en MeshLab (Limpieza del Modelo)

Una vez finalizado el cálculo en COLMAP, se exportó el modelo en formato (.ply) y se importó en MeshLab para refinar la calidad visual del objeto 3D mediante las siguientes fases:

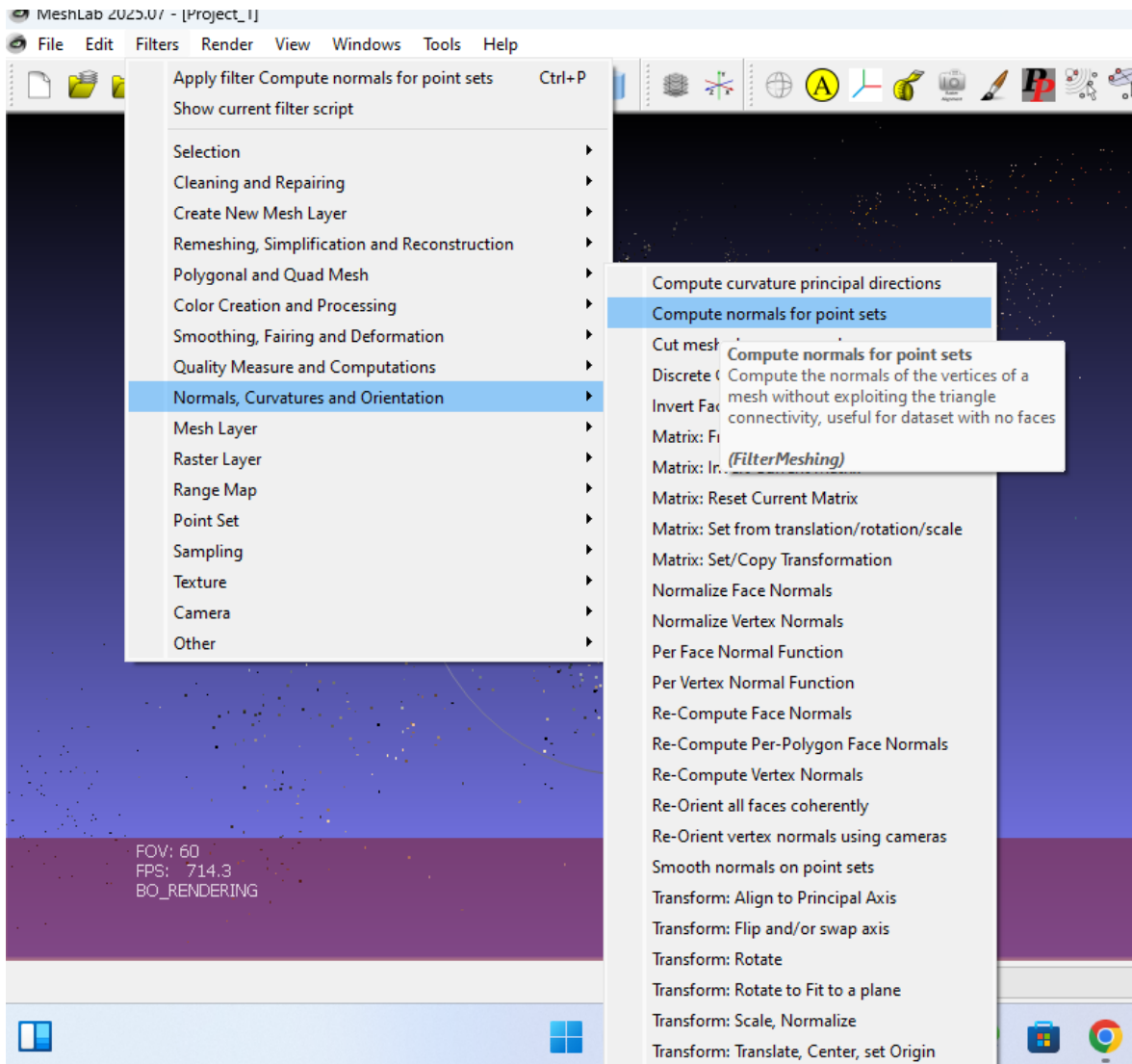
- **Importación:** Lectura directa del archivo (.ply) que conserva las coordenadas espaciales (X, Y, Z) y la información cromática (RGB) de cada punto.



- **Preparar las Normales (Indispensable):**

Paso 1: Antes de crear la superficie, el programa necesita saber hacia dónde "mira" cada punto.

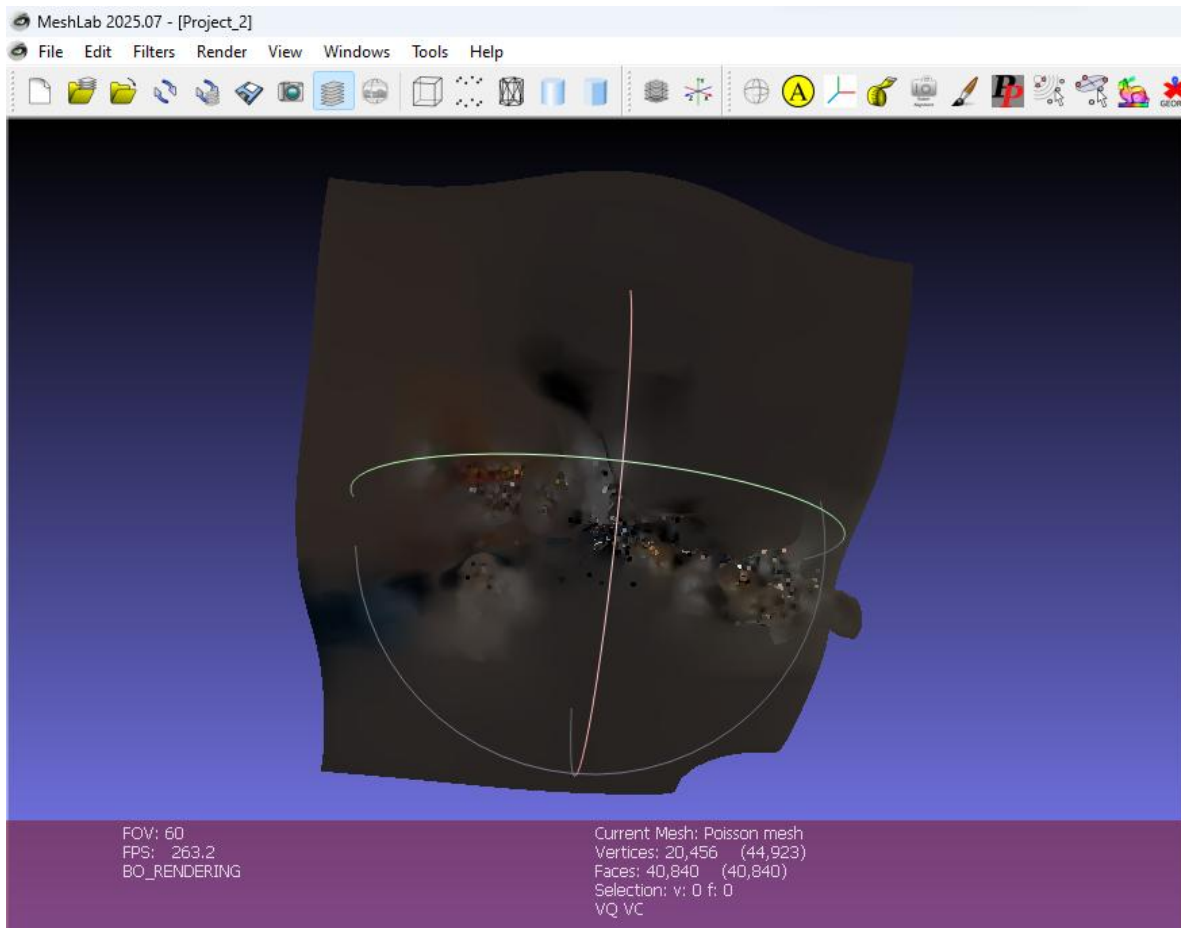
Ve al menú superior: Filters → Normals, Curvatures and Orientation → Compute normals for point sets.



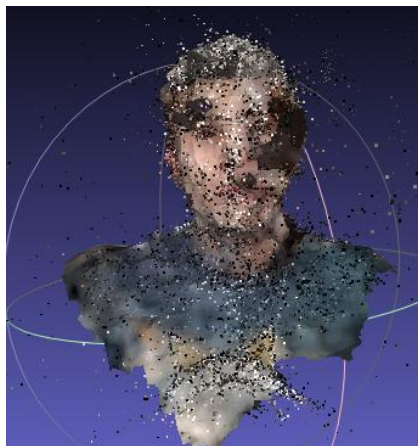
- **Crear la Superficie (Screened Poisson)**

Paso 2.: Ahora se creara la malla sólida.

Ve a: Filters → Remeshing, Simplification and Reconstruction → Surface Reconstruction: Screened Poisson.



- **Visualización:** Ajuste de las propiedades de renderizado para evaluar la calidad estética del modelo desde las vistas principales (frente, perfil, planta) verificando la fidelidad del color heredado de las fotos originales.



5. Resultados y Capturas de Pantalla

1. **Captura de COLMAP:** Vista del modelo 3D rodeado por las pirámides virtuales de color que representan la trayectoria de la cámara del Redmi A2 durante la grabación de 360°.
2. **Captura Comparativa en MeshLab (Antes y Después):** Imagen comparativa que demuestre la nube de puntos cruda con el ruido ambiental circundante frente al modelo final de la persona completamente limpio, aislado y optimizado.



6 conclusiones

- **Éxito de la Reconstrucción:** La reconstrucción tridimensional resultó bien, logrando un modelo digital final donde los rasgos principales y la silueta de la persona son perfectamente reconocibles. Esto fue posible gracias al alto volumen de fotogramas empleados (entre 500 y 600 imágenes), lo que proporcionó suficiente densidad y solapamiento de datos para los algoritmos de fotogrametría.
- **Limitaciones de Hardware y Desempeño:** Al no contar con una tarjeta gráfica dedicada NVIDIA que permitiera el uso de la tecnología **CUDA (GPU)**, el procesamiento de los datos dependió enteramente del procesador principal (CPU). Esto se tradujo en un incremento considerable en los tiempos de cómputo para la alineación de cámaras, provocando además una alta demanda de recursos que llevó a la saturación temporal y fatiga térmica de los equipos utilizados.
- **Eficiencia del Formato (.ply):** Se destaca la ventaja técnica de exportar el resultado final en el formato de polígonos y puntos **(.ply)**. Este archivo encapsula eficientemente toda la información geométrica (X, Y, Z) y cromática (RGB) de la escena. Esto permite almacenar, visualizar y transferir el modelo 3D optimizado a otros colaboradores de manera ágil y compacta, eliminando por completo la necesidad de compartir los pesados archivos de video o fotogramas originales.